

Rapport : Gestion des clés USB par SNMP Trap

LATTARD Florian

1. Choix Techniques

Pour garantir une détection en temps réel sans consommer de ressources par un polling, j'ai configuré une règle udev. Cela permet d'intercepter instantanément les événements du noyau Linux lors de l'insertion d'un matériel.

La règle *usb_detect.rules* doit être placée dans le répertoire */etc/udev/rules.d/*

Voici la règle :

```
ACTION=="add", SUBSYSTEM=="block", ENV{ID_BUS}=="usb", ENV{DEVTYPE}=="disk",  
IMPORT{builtin}=="usb_id", RUN+="/usr/local/bin/usb_alert.sh $devpath"
```

- ACTION=="add" : Se déclenche uniquement au branchement d'un périphérique.
- SUBSYSTEM=="block" : Filtre sur les périphériques de stockage de données.
- ENV{ID_BUS}=="usb" : Cible uniquement le bus USB.
- ENV{DEVTYPE}=="disk" : Sélectionne le disque entier et pas ses partitions.
- IMPORT{builtin}="usb_id" : Extraît les propriétés d'identification de l'appareil.
- RUN+="/usr/local/bin/usb_alert.sh \$devpath" : Exécute le script en lui transmettant le chemin du périphérique dans le dossier système.

Dès le branchement du périphérique, la règle udev exécute le script *usb_alert.sh* (placé dans */usr/local/bin/*)

Le fonctionnement du script dans l'ordre :

Dès le branchement du périphérique, la règle udev exécute le script *usb_alert.sh* (placé dans */usr/local/bin/*), qui analyse le périphérique de stockage USB. Le script récupère d'abord le chemin système de l'appareil via le paramètre \$1 et entre dans une boucle while pour remonter l'arborescence dossier par dossier à l'aide de *dirname* ; dès qu'il détecte le fichier *authorized*, il s'interrompt (*break*) et extrait avec *basename* l'identifiant unique du port USB physique parent. Ensuite, il rassemble le constructeur, le modèle et le numéro de série extraits par udev en appliquant une sécurité par la syntaxe *\${VAR:-ALTERNATIVE}* (qui bascule sur l'ID numérique ou sur la mention "SANS_..." si la donnée est absente) car j'ai vu que certaines clé USB peuvent ne pas avoir certains de ces champs, fusionnant le tout dans la variable *USB_ACTUEL* pour obtenir une ligne normalisée au format Fabricant Modèle NuméroDeSérie. Enfin, il utilise la commande *grep* pour rechercher une correspondance stricte sur une ligne entière dans le fichier */etc/usb_security/whitelist.txt* : si la clé y figure, il laisse l'accès et inscrit une ligne horodatée *ALLOWED* dans le journal */var/log/usb_security.log*, alors que si ça ne correspond pas il déclenche instantanément l'envoi d'un SNMP Trap pour alerter le SOC, injecte la valeur 0 dans le fichier *authorized* du port parent pour couper la communication matérielle au niveau du noyau Linux, et inscrit une ligne horodatée *UNALLOW & BLOCKED* dans le journal log.

Plutôt que de démonter la clé (trop lent), le script écrit 0 dans le fichier système. L'arrêt de l'alimentation logique du port au niveau du noyau bloque l'accès aux données avant toute interaction avec l'espace utilisateur.

2. Tests de Validation

Pour vérifier le blocage et qu'il soit en temps réel, j'ai fait des tests en exécutant en parallèle la commande `watch -n 0.1 lsblk` et la lecture des log via `tail -f /var/log/usb_security.log`

- Test 1 : Clé USB Whitelistée

Branchement de la clé USB whitelistée dans le fichier `/etc/usb_security/whitelist.txt`. Le périphérique est détecté et affiché par `lsblk`. Le fichier de log affiche l'autorisation : `[DATE] - ALLOWED usbX: Kingston DataTraveler...`

- Test 2 : Clé USB Non Whitelistée

Branchement de la clé non whitelistée. Sur la commande `watch lsblk` on peut voir la clé apparaître puis disparaître instantanément. Le fichier de log affiche le blocage : `[DATE] - UNALLOW & BLOCKED usbX: ....`

3. Limites de fonctionnement

-Un attaquant peut utiliser une clé programmable (comme une Rubber Ducky, un Flipper Zero ou un microcontrôleur Teensy) et y programmer le même constructeur, modèle et numéro de série qu'une clé whitelistée. Le système verra la clé comme légitime et lui donnera l'accès.

-Si un attaquant modifie le micrologiciel d'une clé USB pour qu'elle se déclare comme un clavier, ma règle `udev` l'ignorera complètement. L'attaquant ne sera donc pas bloqué.

-Le blocage via le fichier `authorized` du noyau est temporaire. Si la machine redémarre alors que la clé USB non autorisée est branchée au port, le noyau Linux va réinitialiser par défaut la valeur de tous les ports à 1 (autorisé) lors de la phase de boot. Si les règles `udev` ne se chargent pas assez tôt au démarrage, la clé non autorisée sera lue avant le script.

-Le langage Bash est interprété plus ou moins rapidement. Sur un système industriel ancien (CPU faible, ressources limitées), ce traitement pourrait possiblement dépasser les 200 ms exigées, laissant du temps à l'attaquant.

-Le script met la valeur `"SANS_SERIE"` si le numéro de série est absent. Si un attaquant branche une clé USB bas de gamme ou modifiée qui ne renvoie aucun numéro de série, et qu'une clé légitime similaire (sans série) a été enregistrée par erreur dans la whitelist, la clé de l'attaquant sera acceptée.

-Si il y a un problème réseau l'alerte `SNMO Trap` ne sera pas envoyée au SOC.

-Si une clé USB non autorisée est branchée sur un hub, l'intégralité du hub USB sera bloquée, coupant instantanément tous les autres appareils (clavier, souris, autre clé) qui y sont connectés.